

Trabalho de Banco de dados: MySQL

Observe a seguir todos os comandos utilizados no desenvolvimento do trabalho.

Repositório do trabalho: <https://github.com/Jcvanz/ENTRA21-Banco-de-Dados-SQL/tree/main>

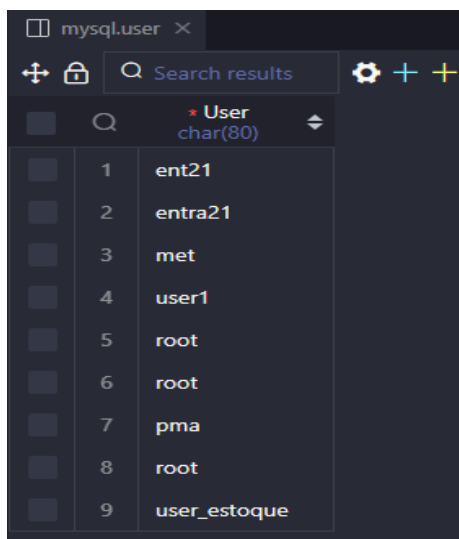
- Criando o usuário e dando as permissões para ele.

```
## Criando o database
▷ Execute
CREATE DATABASE Estoque;

## Criando o usuário
▷ Execute
CREATE USER 'user_estoque'@'localhost' IDENTIFIED BY '1234';

## Concedendo as permissões para o usuário
▷ Execute
GRANT ALL ON * . * TO 'user_estoque'@'localhost';

## Comandao para ver se foi criado o usuário
▷ Execute
SELECT user FROM mysql.user;
```



	User char(80)
1	ent21
2	entra21
3	met
4	user1
5	root
6	root
7	pma
8	root
9	user_estoque

- Criando as tabelas e usando o ALTER TABLE para adicionar as Foreign Keys nas tabelas Stock_Entry e Stock_Exits.

```

## criar tabela produtos
> Execute
create table Products (
id_product int auto_increment,
product_name varchar(45) not null,
desc_product varchar(150) not null,
unit_price decimal(10,2) not null,
primary key(id_product));

## criar tabela entradas
> Execute
create table Stock_Entry (
id_entry int auto_increment,
id_produto int not null,
qtnProduct_entry int not null,
date_entry date not null,
primary key(id_entry));

## criar tabela saidas
> Execute
create table Stock_Exits (
id_exit int auto_increment,
id_produto int not null,
qtnProduct_exits int not null,
date_exit date not null,
primary key(id_exit));

## Alterando a tabela stock_exits para acrescentar a FOREIGN KEY id_produto (referente a tabela products)
> Execute
ALTER TABLE Estoque.stock_exits
ADD CONSTRAINT id_product
FOREIGN KEY (id_produto)
REFERENCES estoque.products (id_product);

## Alterando a tabela stock_entry para acrescentar a FOREIGN KEY id_produto (referente a tabela products)
> Execute
ALTER TABLE estoque.stock_entry
ADD CONSTRAINT id_produto
FOREIGN KEY (id_produto)
REFERENCES estoque.products (id_product);

```

- Inserindo os produtos na tabela products através do comando INSERT INTO.

```

## Inserindo produtos na tabela products, através do INSERT INTO
Execute
INSERT INTO estoque.products (product_name, unit_price, desc_product) VALUES
('Tênis ADIDAS Masc', 150.99, 'Tênis disponível nas cores: azul, branco, vermelho, amarelo'),
('Tênis ADIDAS Fem', 170.99, 'Tênis disponível nas cores: azul, branco, vermelho, amarelo') ,
('Tênis NIKE Masc', 200.99, 'Tênis disponível nas cores: azul, branco, vermelho, amarelo'),
('Tênis NIKE Fem', 50.99, 'Tênis disponível nas cores: azul, branco, vermelho, amarelo'),
('Tênis UMBRO Masc', 100.99, 'Tênis disponível nas cores: azul, branco, vermelho, amarelo'),
('Tênis VANS Masc', 124.99, 'Tênis disponível nas cores: azul, branco, vermelho, amarelo'),
('Tênis VANS Masc', 130.99, 'Tênis disponível nas cores: azul, branco, vermelho, amarelo'),
('Tênis PUMA Masc', 308.99, 'Tênis disponível nas cores: azul, branco, vermelho, amarelo'),
('Tênis PUMA Fem', 50.99, 'Tênis disponível nas cores: azul, branco, vermelho, amarelo'),
('Tênis ADIDAS Masc', 260.99, 'Tênis disponível nas cores: azul, branco, vermelho, amarelo'),
('Tênis NIKE Masc', 400.99, 'Tênis disponível nas cores: azul, branco, vermelho, amarelo');

## Utilizando o SELECT para ver se foi adicionado corretamente
Execute
SELECT * FROM products;

```

products					
Search results					
Cost: 9ms < 1 > Total 11					
		id_produc int(11)	* product_name varchar(45)	* desc_product varchar(150)	* unit_price decimal(10,2)
	1	1	Tênis ADIDAS Masc	Tênis disponível nas cores: a	150.99
	2	2	Tênis ADIDAS Fem	Tênis disponível nas cores: a	170.99
	3	3	Tênis NIKE Masc	Tênis disponível nas cores: a	200.99
	4	4	Tênis NIKE Fem	Tênis disponível nas cores: a	50.99
	5	5	Tênis UMBRO Masc	Tênis disponível nas cores: a	100.99
	6	6	Tênis VANS Masc	Tênis disponível nas cores: a	124.99
	7	7	Tênis VANS Masc	Tênis disponível nas cores: a	130.99
	8	8	Tênis PUMA Masc	Tênis disponível nas cores: a	308.99
	9	9	Tênis PUMA Fem	Tênis disponível nas cores: a	50.99
	10	10	Tênis ADIDAS Masc	Tênis disponível nas cores: a	260.99
	11	11	Tênis NIKE Masc	Tênis disponível nas cores: a	400.99

- Inserindo os produtos na tabela Stock_Entry através do comando INSERT INTO.

```
## Inserindo produtos na tabela stock_entry, através do INSERT INTO
> Execute
INSERT INTO estoque.stock_entry (id_produto, qtnProduct_entry, date_entry) VALUES
(1, 10, '2021-02-10'),
(2, 10, '2023-11-19'),
(3, 8, '2020-12-13'),
(4, 5, '2021-01-06'),
(5, 1, '2021-11-08'),
(6, 3, '2020-05-10'),
(7, 9, '2023-11-21'),
(8, 5, '2022-01-15'),
(9, 7, '2020-11-10'),
(10, 8, '2022-07-10'),
(11, 10, '2021-11-10');

## Utilizando o SELECT para ver se foi adicionado corretamente
> Execute
SELECT * FROM stock_entry; 3ms
```

stock_entry X					
Search results					
		id_entry int(11)	* id_produto int(11)	* qtnProduct_entry int(11)	* date_entry date
	1	1	1	10	2021-02-10
	2	2	2	10	2023-11-19
	3	3	3	8	2020-12-13
	4	4	4	5	2021-01-06
	5	5	5	1	2021-11-08
	6	6	6	3	2020-05-10
	7	7	7	9	2023-11-21
	8	8	8	5	2022-01-15
	9	9	9	7	2020-11-10
	10	10	10	8	2022-07-10
	11	11	11	10	2021-11-10

- Inserindo os produtos na tabela Stock_Exits através do comando INSERT INTO.

```

## Inserindo produtos na tabela stock_entry, através do INSERT INTO
> Execute
INSERT INTO estoque.stock_exits (id_produto, qtnProduct_exits, date_exit) VALUES
(1, 1, NOW()),
(2, 1, NOW()),
(3, 1, NOW()),
(4, 2, NOW()),
(5, 1, NOW()),
(6, 1, NOW()),
(7, 1, NOW()),
(8, 1, NOW()),
(9, 1, NOW()),
(10, 1, NOW()),
(11, 2, NOW());

## Utilizando o SELECT para ver se foi adicionado corretamente
> Execute
SELECT * FROM stock_exits; 4ms

```

stock_exits					
		id_exit int(11)	* id_produto int(11)	* qtnProduct_exits int(11)	* date_exit date
	1	1	1	1	2023-11-23
	2	2	2	1	2023-11-23
	3	3	3	1	2023-11-23
	4	4	4	2	2023-11-23
	5	5	5	1	2023-11-23
	6	6	6	1	2023-11-23
	7	7	7	1	2023-11-23
	8	8	8	1	2023-11-23
	9	9	9	1	2023-11-23
	10	10	10	1	2023-11-23

- Utilizando o UPDATE para alterar a quantidade de produtos

```
## Utilizando o UPDATE para alterar a quantidade de produtos
▷ Execute
UPDATE stock_exits
SET amount_product = 2
WHERE id_produto = 1;
```

- Deletando registro de saída através do comando DELETE

```
## Deletando o último registro de saída
▷ Execute
DELETE FROM stock_exits
WHERE id_exit = 11;
```

- Listando as operações de entrada em um determinado período, usando SELECT FROM para passar o caminho, WHERE para indicar a coluna e BETWEEN AND para selecionar as datas

```
## Listando as operações de entrada em um determinado período
▷ Execute
SELECT id_produto, date_entry
FROM stock_entry
WHERE date_entry BETWEEN '2023-10-20' AND '2023-11-22';
```

- Listando uma operação de saída específico com o comando SELECT FROM

```
## Mostrando as operações de saída de um produto específico
▷ Execute
SELECT *
FROM stock_entry
WHERE date_entry = '2023-11-19';
```

- Mostrando a quantidade de estoque, após fazer o cálculo da quantidade de entrada menos a quantidade de saída e fazendo o cálculo da quantidade que ainda tem em estoque vezes o valor de cada tênis;
- SELECT FROM para selecionar os itens;
- INNER JOIN para comparar cada linha e encontrar as linhas que cumpre os requisitos passados;
- GROUP BY para combinar os registros com os valores idênticos da lista de campos especificada;
- AS para atribuir um “apelido” para aquele comando

```

/*
Mostrando a quantidade de estoque, após fazer o cálculo da quantidade de entrada menos a quantidade de saída e
fazendo o cálculo da quantidade que ainda tem em estoque vezes o valor de cada tênis
*/
Execute
SELECT
    tbprodutos.id_product,
    tbprodutos.product_name,
    tbprodutos.unit_price,
    tbentradas.total_entrada,
    tbsaidas.total_saida,
    (tbentradas.total_entrada - tbsaidas.total_saida) as Quantidade_estoque,
    ((tbentradas.total_entrada - tbsaidas.total_saida) * tbprodutos.unit_price) as valor_Estoque
FROM estoque.products tbprodutos
INNER JOIN (SELECT id_producto, SUM(qtnProduct_entry) as total_entrada
FROM estoque.stock_entry
GROUP BY id_producto) tbentradas
ON tbprodutos.id_product = tbentradas.id_producto
INNER JOIN (SELECT id_producto, SUM(qtnProduct_exits) as total_saida
FROM estoque.stock_exits
GROUP BY id_producto) tbsaidas
ON tbprodutos.id_product = tbsaidas.id_producto;

```

Result								
Search results								
Cost: 6ms < 1 > Total 10								
		* id_product int(11)	* product_name varchar(45)	* unit_price decimal(10,2)	total_entrada newdecimal	total_saida newdecimal	Quantidade_estoque newdecimal	valor_Estoque newdecimal
1	1	1	Tênis ADIDAS Masc	150.99	10	1	9	1358.91
2	2	2	Tênis ADIDAS Fem	170.99	10	1	9	1538.91
3	3	3	Tênis NIKE Masc	200.99	8	1	7	1406.93
4	4	4	Tênis NIKE Fem	50.99	5	2	3	152.97
5	5	5	Tênis UMBRO Masc	100.99	1	1	0	0.00
6	6	6	Tênis VANS Masc	124.99	3	1	2	249.98
7	7	7	Tênis VANS Masc	130.99	9	1	8	1047.92
8	8	8	Tênis PUMA Masc	308.99	5	1	4	1235.96
9	9	9	Tênis PUMA Fem	50.99	7	1	6	305.94
10	10	10	Tênis ADIDAS Masc	260.99	8	1	7	1826.93

- Identifique produtos com estoque abaixo de um nível mínimo e fazendo os cálculos para mostrar o total de entrada e de saída de cada produto.

```

/*
  Identifique produtos com estoque abaixo de um nível mínimo.
  Fazendo os cálculos para mostrar o total de entrada e de saída de cada produto.
*/
Execute
SELECT
  tbprodutos.id_product,
  tbprodutos.product_name,
  tbentradas.total_entrada,
  tbsaidas.total_saida,
  (tbentradas.total_entrada - tbsaidas.total_saida) as Estoque_baixo
FROM estoque.products tbprodutos
INNER JOIN (SELECT id_producto, SUM(qtnProduct_entry) as total_entrada
  FROM estoque.stock_entry
  GROUP BY id_producto) tbentradas
  ON tbprodutos.id_product = tbentradas.id_producto
INNER JOIN (SELECT id_producto, SUM(qtnProduct_exits) as total_saida
  FROM estoque.stock_exits
  GROUP BY id_producto) tbsaidas
  ON tbprodutos.id_product = tbsaidas.id_producto
WHERE (tbentradas.total_entrada - tbsaidas.total_saida) < 5; 8ms

```

Result						
Search results						
		id_product int	product_name varchar	total_entrada newdecimal	total_saida newdecimal	Estoque_baixo newdecimal
1	4		Tênis NIKE Fem	5	2	3
2	5		Tênis UMBRO Masc	1	1	0
3	6		Tênis VANS Masc	3	1	2
4	8		Tênis PUMA Masc	5	1	4

A parte mais desafiadora que encontrei foi para resolver os últimos dois desafios, os quais nunca tinha feito antes. Consegui resolver através da explicação do professor Samuel na aula de quinta-feira (23/11).

Para o controle de estoques, o MySQL cede uma plataforma muito boa, oferecendo as ferramentas necessárias para a construção do banco de dados de maneira muito segura.